TaskFlow

Modern Django CRUD with SQLite

A complete local portfolio project with a responsive dashboard, user accounts, per-user tasks, and a REST API.

Features

Create tasks with title, description, status, priority and due date. Read them in a dashboard or detail page. Update them with validated forms. Delete them through a confirmation page. Search, filter and paginate your task list.

Database included through migrations

SQLite requires no separate server. Running migrate creates db.sqlite3 and all required tables. The archive contains the model and migration files rather than a live database containing accounts.

Architecture

Browser -> Django URL routing -> authenticated view -> ModelForm / Django ORM -> SQLite -> HTML response. REST clients use Django REST Framework serializers and viewsets with the same Task model.

Included

Python source, responsive CSS and templates, database migration, fictional-data seeding command, six automated tests, README, and this guide.

1. Setup and run

Use Python 3.10 or newer. Extract the ZIP and open a terminal inside django-taskflow.

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Open http://127.0.0.1:8000/signup/ and create your account. Sign in using that username and password. No default credentials are included.

Optional sample data

```
python manage.py seed_demo YOUR_USERNAME
```

Run this in another activated terminal after account creation. It adds five fictional tasks without changing passwords.

Admin and verification

```
python manage.py createsuperuser
python manage.py test
python manage.py check
```

The superuser can access /admin/ and manage all tasks. Ordinary users access only their own tasks.

2. CRUD walkthrough

Create

Select + New task. Enter a title, description, status, priority and optional date. Save task. Django validates the form, assigns the signed-in user as owner, and writes the record to SQLite.

Read

The dashboard shows task totals and a paginated list. Search by title or description and filter by status. Click a task title for its detail page. Dashboard counts always cover all your tasks.

Update

Select Edit, change the fields and save. The same database row is updated and its updated_at timestamp changes.

Delete

Select Delete to open a confirmation page. Confirm with the Delete task button. Viewing the confirmation page does not remove data; the POST request does.

REST API

After signing in, open /api/tasks/ in the same browser to use the browsable API. GET lists tasks, POST creates, GET /api/tasks/ID/ reads one, PUT replaces, PATCH updates selected fields, and DELETE removes.

```
{"title": "Learn Django ORM",
 "status": "progress", "priority": "high",
 "description": "Practice CRUD queries",
 "due_date": "2026-12-01"}
```

API options: ?search=Django, ?status=done, ?page=2, and ?ordering=due_date. The API uses session cookies; unsafe requests require a CSRF token. Use the browsable forms for the simplest demo.

3. Database and GitHub

Task schema

id: primary key. owner: foreign key to Django user. title: up to 150 characters. description: optional text up to 3,000 characters. status: todo / progress / done. priority: low / medium / high. due_date: optional date. created_at and updated_at: automatic timestamps. An owner/status index supports filtering.

Why ownership matters

Every normal web and API queryset filters by the signed-in owner. Other users' IDs return 404. Owner cannot be supplied through forms or the serializer. Login and CSRF middleware protect web actions. Trusted administrators have broader access.

Upload to GitHub

Create an empty repository named django-taskflow. Extract the archive and upload its contents so README.md and manage.py appear at the repository root. Prefer Git to preserve hidden files such as .gitignore.

```
git init
git add .
git commit -m "Add Django CRUD project"
git branch -M main
git remote add origin \
 https://github.com/praveenkumarmandula/django-taskflow.git
git push -u origin main
```

Never upload your local db.sqlite3 or real credentials. The ZIP excludes them. The README contains setup and extension ideas.

Tests and deployment scope

Six tests cover HTML CRUD, API CRUD and validation, authentication, user isolation, search and CSRF enforcement. This is a local development project. Before shared deployment, configure a secret key, DEBUG off, allowed hosts, HTTPS, secure cookies, static serving and a production server.

Official references

https://docs.djangoproject.com/en/5.2/
https://www.django-rest-framework.org/api-guide/viewsets/